

Inferential Biases in Constrained Generation with Autoregressive Models

François Pachet
LIP6, Sorbonne Université
pachet@gmail.com

Pierre Roy
Soundtrap
roypie@gmail.com

Abstract

Large language and music models are increasingly used for constrained generation: rhyming lines, fixed meter, inpainting or infilling, positional endings, and other global form requirements. These systems often perform strikingly well, but the induced procedures are usually not exact conditioning of the underlying autoregressive model. This creates a hidden inferential bias, distinct from the better-known notion of bias inherited from the training set: samples are distorted relative to the true constrained distribution, with no generic guarantee of complete coverage of the admissible solution space or of correct conditional probabilities over valid completions. We formalize several exact inference tasks for autoregressive models and prove corresponding hardness results. For succinctly represented autoregressive models whose next-token probabilities are computable in polynomial time, exact sentence-level maximum a posteriori (MAP) decoding is NP-hard. This hardness persists under unary constraints and under a fixed metrical-count special case. Even approximate constrained optimization remains hard in a strong sense: for every fixed $\varepsilon < 1$, finding a feasible fixed-length completion whose average surprisal is within ε bits per token of optimum is NP-hard. On the sampling side, exact conditioned normalization is #P-hard even for regular constraints such as fixed-length terminal events. Unlike finite-state Markov models, general autoregressive models do not admit a bounded-state dynamic program for these tasks. These results formalize a standard claim in the neural decoding literature: local autoregressive sampling is easy, whereas exact decoding, exact conditioning, and even sub-one-bit-per-token approximation under global form constraints are computationally intractable in general.

1 Introduction

Large language models are now routinely used for constrained generation. Users ask them to write rhyming lines, continue a poem in a prescribed meter, fill in missing text, complete a partially specified sentence, enforce a suffix or a letter pattern at a given position, or satisfy combinations of such requirements. Similar uses appear in symbolic music, where sequence models are asked to inpaint missing passages or complete a score subject to formal constraints. Empirically, these systems often perform strikingly well. They produce many valid-looking constrained outputs, and they often give the impression that formal control is largely solved.

A contemporary text example, motivated by a presentation by Constantinos Daskalakis at Festum-Pi 2025 [24], illustrates the problem. If one asks ChatGPT to rewrite the beginning of Genesis using only words starting with the letter *A*, the result is convincing (bold letters indicate mistakes):

At an ancient absence, allness awaited.
Above, abyssal airs and ash.
Around, a formless ambiguity.
And an august animation, afloat above abysmal aquifers.

The constraint is almost perfectly satisfied: each generated word must come from a filtered vocabulary. However, if one asks for sentences that begin and end with the same letter, one gets an output such as:

Ages ago, all was ash and aura.
 Before being, blackness brooded beneath.
 Chaos curled, concealed in cosmic dark.
 Deep drifted, dim and dreamlike, dead.
 ...
 Judgment joined light from lingering gloom, judgment.

Though the second request sounds simpler, it actually exposes the distinction at the heart of this paper. The first constraint can be checked token by token from a filtered vocabulary. The second is global: local choices are coupled to the space of future completions, because the validity of a partial prefix depends on how the sentence will eventually end.

The same example also gives an intuition for our approximation-hardness result. Suppose one weakens the request and asks only for a Genesis-like continuation of exactly L tokens, to be judged by average log-probability among completions of that length. One might expect this relaxed objective to ask only for a *pretty good* constrained line rather than the exact optimum. Yet in the broad autoregressive regime studied here, even guaranteeing a completion within, say, half a bit per token of the best valid fixed-length continuation is NP-hard. So the obstacle is not limited to exact satisfaction or exact MAP decoding; it already appears at the level of near-optimal constrained continuations.

One point deserves clarification up front. A prompt that states a constraint does not by itself reveal which mechanism is used to satisfy it. A deployed system may rely on prompting alone, on explicit constrained decoding, on reranking, on rejection sampling, or on a specialized infilling architecture. Throughout this paper, we therefore use *constrained generation* in the broad sense: a user or application specifies a formal property E of complete outputs, and we compare the resulting generation procedure with exact conditioning of a fixed underlying autoregressive model on the event E .

This contrast is easy to miss in practice, because current systems often produce striking constrained outputs. But that practical success can be misleading. When an autoregressive model is used in this way, the resulting procedure is usually not exact conditioning of the original model on the requested constraint. Instead, practical systems rely on heuristic search, reranking, local reweighting, rejection sampling, or separately trained infilling architectures. The resulting samples are therefore biased relative to the exact conditional law of the fixed underlying autoregressive model. More importantly, one generally does not know which part of the admissible solution space is actually explored, whether all valid completions remain reachable, or whether feasible completions are produced with the correct conditional probabilities.

This is a different notion of bias from the one usually discussed for large language models. There the term typically refers to skew inherited from the training distribution or reflected in the generated content [2]. Here the bias is computational: with the pretrained model held fixed, approximate constrained-generation procedures distort the exact conditional distribution induced by the requested constraint.

The results of this paper explain why. The issue is not that such constraints are linguistically or musically exotic. On the contrary, many of them are simple to state and even regular or additive in structure. The difficulty comes from the mismatch between local left-to-right generation and global exact satisfaction of a property of the whole completed sequence.

An autoregressive model with parameters θ defines, for each finite sequence $x = x_{1:n}$,

$$P_\theta(x_{1:n}) = \prod_{i=1}^n P_\theta(x_i \mid x_{1:i-1}).$$

This factorization suggests computational simplicity. For unconstrained generation, that is correct: one samples sequentially from the next-token conditionals. But sequence-level optimization or conditioning is a different problem. Given access to the local conditionals, one may ask for the globally most probable complete sequence,

$$x^* \in \arg \max_{x \text{ complete}} P_\theta(x),$$

or for exact sampling from the model conditioned on some global event.

For finite-state Markov models, such tasks are solved by dynamic programming because the relevant dependence on the past is summarized by a bounded state [11]. General autoregressive models do not enjoy this property. Their next-token distribution may depend on the entire prefix through an unbounded contextual representation. As a result, the factorization of the probability does not imply a Viterbi-style algorithm for recovering the most probable sequence, nor an efficient forward-backward style algorithm for exact conditioning.

This distinction is reflected in practice. In the neural sequence generation literature, exact MAP decoding is treated as computationally difficult, and approximate methods such as greedy search, beam search, and reranking are used instead [4, 3, 1]. A complementary line of work studies constrained or controlled decoding directly, for example through lexically constrained beam search and logical constrained decoding [7, 8, 9]. Recent work on grammar-aligned decoding makes a closely related point for grammar constraints, showing that constrained decoding can distort the underlying model distribution unless one corrects for future feasibility [10]. Yet these issues are usually framed algorithmically rather than complexity-theoretically. The purpose of this note is to make a family of precise hardness statements explicit.

Our contribution is elementary and conceptually clarifying. We prove that exact sentence-level MAP decoding is NP-hard for a natural class of succinctly represented autoregressive models. The hardness persists under unary constraints and under a fixed metrical-count special case. Even approximate constrained generation remains hard: under the fixed-length regular constraint, achieving any fixed additive approximation strictly below one bit per token in average surprisal is NP-hard. On the sampling side, we prove that exact conditioned normalization is already #P-hard for regular constraints as simple as “having fixed length L and ending in `eos`.” This perspective explains why current constrained-generation systems can be useful and impressive while still failing to deliver exact conditional sampling: approximate search may distort the conditional weights on valid completions, or it may distort the support itself by making some valid completions unreachable.

The main statements can be read as a short map:

- exact sentence-level MAP decoding is NP-hard;
- this NP-hardness persists under unary position-wise constraints and a fixed metrical-count special case;
- even approximating the best feasible fixed-length completion within any fixed additive constant smaller than one bit per token in average surprisal is NP-hard;
- exact conditioned normalization for fixed-length terminal events is #P-hard, even though the constraint language is regular;
- exact inpainting by causal left-to-right renormalization of a fixed pretrained model inherits this #P-hardness;
- by contrast, in first-order Markov models, exact sampling with correct conditional probabilities is known for regular constraints, including fixed-position unary constraints, and for Markov constraints [23].

2 Setup

Let Σ be a finite vocabulary containing a distinguished end-of-sequence symbol **eos**. An autoregressive model assigns probabilities to finite complete sequences through conditionals of the form

$$P_\theta(x_{1:n}) = \prod_{i=1}^n P_\theta(x_i \mid x_{1:i-1}),$$

with the convention that generation terminates as soon as **eos** is produced. We therefore write $\mathcal{C}$ for the set of complete sequences

$$\mathcal{C} = \{x_{1:n} \in \Sigma^n : n \geq 1, x_n = \mathbf{eos}, \text{ and } x_i \neq \mathbf{eos} \text{ for } i < n\}.$$

Definition 1 (Sentence-level MAP decoding). *Given an autoregressive model P_θ , the sentence-level maximum a posteriori (MAP) problem is*

$$x^* \in \arg \max_{x \in \mathcal{C}} P_\theta(x).$$

In the present setting it simply denotes the most probable complete sequence under the model.

To obtain a genuine complexity statement, the model itself must be part of the input. We therefore consider a class $\mathcal{M}$ of *succinctly represented autoregressive models* such that:

- (i) each model $P_\theta \in \mathcal{M}$ has a finite description whose size is polynomial in the instance size;
- (ii) for every prefix $x_{1:i-1}$ and every token $a \in \Sigma$, the conditional probability $P_\theta(a \mid x_{1:i-1})$ is a rational number whose exact value is computable in polynomial time from the model description and the prefix.

This captures the natural setting in which local next-token evaluation is efficient, while global optimization or exact conditioning over complete sequences can still be hard.

This class is intended to capture the regime of modern pretrained autoregressive models used in practice: the model description is succinct, and next-token probabilities are efficiently evaluable from a prefix. In that sense, the results are not about learning but about exact inference for a fixed pretrained model. They are worst-case statements. They do not say that every prompt, every constraint, or every instance is hard, nor that approximate constrained-generation methods cannot work well in practice. They say that no generic exact polynomial-time reduction from global constrained inference to ordinary left-to-right sampling exists across this broad autoregressive model class.

3 Exact MAP decoding

Theorem 1. *Exact sentence-level MAP decoding over succinctly represented autoregressive models is NP-hard.*

Proof. We reduce from SAT. Let φ be a CNF formula over Boolean variables $v_1, \dots, v_m$. From φ we construct, in polynomial time, an autoregressive model P_φ over the alphabet

$$\Sigma = \{0, 1, b_0, b_1, \mathbf{eos}\}.$$

The constructed model assigns higher probability to satisfying assignments than to non-satisfying ones, so that any MAP solution encodes a satisfying assignment.

For the first m positions, the model chooses bits uniformly:

$$P_\varphi(x_i = 0 \mid x_{1:i-1}) = P_\varphi(x_i = 1 \mid x_{1:i-1}) = \frac{1}{2}, \quad i = 1, \dots, m.$$

Hence every prefix $a = (a_1, \dots, a_m) \in \{0, 1\}^m$ has probability 2^{-m} and encodes a truth assignment for φ .

At step $m + 1$, the model inspects the assignment encoded by the prefix. If $a \models \varphi$, it emits **eos** with probability 1, so the sequence terminates at length $m + 1$. Otherwise it emits b_0 and b_1 with probabilities $\frac{1}{2}$ each. In the latter case, at step $m + 2$ the model emits **eos** with probability 1.

Therefore,

$$P_\varphi(a \text{ eos}) = 2^{-m} \quad \text{if } a \models \varphi,$$

whereas for each $j \in \{0, 1\}$,

$$P_\varphi(a b_j \text{ eos}) = 2^{-m-1} \quad \text{if } a \not\models \varphi.$$

If φ is satisfiable, every satisfying assignment yields a complete sequence of probability 2^{-m} , and every complete sequence corresponding to a non-satisfying assignment has strictly smaller probability 2^{-m-1} . Hence any MAP-optimal sequence encodes a satisfying assignment.

If φ is unsatisfiable, then every complete sequence has probability exactly 2^{-m-1} .

Thus an exact MAP decoder decides SAT: compute a most probable complete sequence and check whether its $(m + 1)$ st token is **eos**, equivalently whether its first m symbols satisfy φ . The construction is polynomial-size, and each conditional probability is polynomial-time computable because the only nontrivial step is evaluating whether an assignment satisfies a CNF formula. Since SAT is NP-hard, exact sentence-level MAP decoding is NP-hard. $\square$

4 Unary-constrained MAP decoding

We now consider position-wise unary constraints. Let $S_1, \dots, S_n \subseteq \Sigma$ be allowed token sets, and consider the constrained optimization problem

$$x^* \in \arg \max_{x \in \mathcal{C}, |x|=n} P_\theta(x) \quad \text{subject to } x_i \in S_i \text{ for all } i.$$

Corollary 1. *Exact sentence-level MAP decoding under unary position-wise constraints is NP-hard.*

Proof. Using the construction above, impose the unary constraints

$$x_i \in \{0, 1\}, \quad i = 1, \dots, m,$$

and

$$x_{m+1} = \text{eos}.$$

Then the feasible sequences are exactly those of the form $a \text{ eos}$ with $a \in \{0, 1\}^m$. Under the model P_φ ,

$$P_\varphi(a \text{ eos}) = \begin{cases} 2^{-m}, & \text{if } a \models \varphi, \\ 0, & \text{otherwise,} \end{cases}$$

because a non-satisfying assignment cannot emit **eos** at position $m + 1$ in this construction. Hence the optimum constrained probability is positive if and only if φ is satisfiable, and an exact constrained MAP decoder decides SAT by returning an optimal feasible sequence whose probability can then be evaluated in polynomial time. Therefore unary-constrained MAP decoding is NP-hard. In particular, NP-hardness already holds for the fixed-terminal constraint that the sequence end with **eos** at a prescribed position. $\square$

5 Exact conditioning on a terminal token at fixed length

Constraining the last token to be `eos` at a prescribed length L is a special case of unary constraints: one simply takes

$$S_i = \Sigma \setminus \{\text{eos}\} \quad (i < L), \quad S_L = \{\text{eos}\}.$$

However, the associated inference task is different. For MAP decoding, this is just a special unary-constrained optimization problem. For exact sampling, what matters is the normalization constant of the conditioned distribution.

For fixed length L , define

$$Z_L(\theta) = \sum_{x_{1:L-1} \in (\Sigma \setminus \{\text{eos}\})^{L-1}} P_\theta(x_{1:L-1}, \text{eos}),$$

that is, the total probability mass of all sequences of length exactly L ending in `eos`.

Exact sampling from the fixed-length conditional distribution

$$P_\theta(x \mid x \in \mathcal{C}, |x| = L)$$

requires, at each step, access to conditioned continuation masses of this kind.

Theorem 2. *For succinct autoregressive models with polynomial-time next-token evaluation, exact computation of $Z_L(\theta)$ is #P-hard.*

Proof. We reduce from #SAT. Let φ be a CNF formula over Boolean variables $v_1, \dots, v_m$, and let $L = m + 1$. Construct an autoregressive model P_φ over

$$\Sigma = \{0, 1, b_0, b_1, \text{eos}\}$$

as follows. For the first m positions,

$$P_\varphi(x_i = 0 \mid x_{1:i-1}) = P_\varphi(x_i = 1 \mid x_{1:i-1}) = \frac{1}{2}, \quad i = 1, \dots, m.$$

Thus the prefix $x_{1:m}$ encodes a uniformly random truth assignment $a \in \{0, 1\}^m$.

At step $m + 1$, if $a \models \varphi$, the model emits `eos` with probability 1; otherwise it emits b_0 and b_1 with probabilities $\frac{1}{2}$ each, followed by `eos` with probability 1 at step $m + 2$.

Hence,

$$Z_L(P_\varphi) = \sum_{a \in \{0,1\}^m} P_\varphi(a, \text{eos}) = \frac{\#\text{SAT}(\varphi)}{2^m}.$$

Therefore exact computation of $Z_L(\theta)$ yields the number of satisfying assignments of φ . Since #SAT is #P-hard, so is exact computation of $Z_L(\theta)$. $\square$

Corollary 2. *Exact sampling from the fixed-length conditional distribution*

$$P_\theta(x \mid x \in \mathcal{C}, |x| = L)$$

does not reduce to ordinary ancestral sampling in the general autoregressive setting. It requires global conditioned continuation masses whose exact computation is #P-hard.

Remark 1. *The set*

$$R_L = \{x \in \mathcal{C} : |x| = L\} = (\Sigma \setminus \{\text{eos}\})^{L-1} \text{eos}$$

is a regular language. Thus regularity of the constraint language is not sufficient for tractability in the general autoregressive setting considered here.

6 Metrical constraints

A fixed terminal position is a special case of a metrical constraint. Let each token $a \in \Sigma$ be assigned a nonnegative integer syllabic weight $w(a)$, and define the set of metrical sentences of total weight K by

$$\mathcal{S}_K = \left\{ x \in \mathcal{C} : \sum_{i=1}^{|x|} w(x_i) = K \right\}.$$

The corresponding constrained MAP problem is

$$\arg \max_{x \in \mathcal{S}_K} P_\theta(x),$$

and the corresponding normalization constant is

$$Z_K(\theta) = \sum_{x \in \mathcal{S}_K} P_\theta(x).$$

The formal hardness statement proved in this section isolates a fixed metrical-count special case. Namely, the fixed-length terminal constraint studied above is an immediate special case of meter: assign unit weight to every token, that is, $w(a) = 1$ for all $a \in \Sigma$. Then the condition $\sum_i w(x_i) = K$ is exactly the condition that the sentence has length K . This does not exhaust realistic meter, but it already captures the additive nature of the constraint. More generally, additive metrical control is algorithmically nontrivial even in first-order Markov models: Roy and Pachet study meter in finite-length Markov sequences and obtain a specialized pseudo-polynomial algorithm for cumulative-cost metrical constraints [12].

Corollary 3. *Exact MAP decoding under a fixed metrical-count constraint is NP-hard.*

Proof. Take $w(a) = 1$ for all tokens. Then a fixed metrical-count constraint with target $K = L$ is equivalent to the fixed-length terminal constraint at length L . By the fixed-terminal special case of Corollary 1, exact MAP decoding under this unit-weight metrical-count constraint is NP-hard. $\square$

Corollary 4. *For succinct autoregressive models with polynomial-time next-token evaluation, exact computation of the total probability mass of the fixed metrical-count event $\mathcal{S}_K$ is #P-hard.*

Proof. Again take $w(a) = 1$ for all tokens. Then

$$Z_K(\theta) = \sum_{x \in \mathcal{S}_K} P_\theta(x)$$

is exactly the fixed-length terminal normalization constant $Z_L(\theta)$ with $L = K$. The claim therefore follows immediately from the preceding theorem. $\square$

Remark 2. *This reduction is deliberately simple: it shows that a fixed metrical-count special case is already hard in the degenerate unit-syllable setting. A direct reduction from subset-sum or counting subset-sum would match heterogeneous additive meter more closely. The point here is not that all realistic metrical systems collapse to fixed length, but that even this simplest additive case already inherits hardness in the general autoregressive setting, while richer metrical constraints require specialized treatment even for Markov models [12].*

7 Decision version

For completeness, consider the threshold problem.

Definition 2 (MAP-THRESHOLD). ***Input:** a succinct autoregressive model P_θ , an integer n , and a rational threshold τ .*

***Question:** does there exist a complete sequence $x_{1:n} \in \mathcal{C}$ of length n such that*

$$P_\theta(x_{1:n}) \geq \tau?$$

Corollary 5. *MAP-THRESHOLD is NP-complete.*

Proof. Membership in NP is immediate: guess a sequence $x_{1:n}$, compute the n local conditional probabilities, multiply them, and compare the result with τ . NP-hardness follows from the SAT reduction above by taking $n = m + 1$ and $\tau = 2^{-m}$. $\square$

8 Approximation hardness in average surprisal

In practice, candidate constrained completions are often compared by average log-probability rather than raw sentence probability, so as to normalize away trivial length effects. For the fixed-length event

$$R_L = \{x \in \mathcal{C} : |x| = L\},$$

define the minimum average surprisal in bits per token by

$$\text{AS}_L(\theta) = \min_{x \in R_L} \left(-\frac{1}{L} \log_2 P_\theta(x) \right).$$

An additive- ε approximation algorithm for this objective returns some $x \in R_L$ such that

$$-\frac{1}{L} \log_2 P_\theta(x) \leq \text{AS}_L(\theta) + \varepsilon.$$

Theorem 3. *For every fixed constant $0 < \varepsilon < 1$, the additive- ε approximation problem above is NP-hard over succinct autoregressive models with polynomial-time next-token evaluation. This hardness already holds for the regular fixed-length terminal constraint R_L .*

Proof. Fix a constant $0 < \varepsilon < 1$. Choose an integer $c \geq 1$ such that

$$\frac{c}{c+2} > \varepsilon,$$

and reduce from SAT. Let φ be a CNF formula over Boolean variables $v_1, \dots, v_m$. Set $r = cm$ and $L = m + r + 1$. Construct an autoregressive model P_φ over the alphabet

$$\Sigma = \{0, 1, p, b_0, b_1, \text{eos}\}$$

as follows.

For the first m positions, the model chooses bits uniformly:

$$P_\varphi(x_i = 0 \mid x_{1:i-1}) = P_\varphi(x_i = 1 \mid x_{1:i-1}) = \frac{1}{2}, \quad i = 1, \dots, m.$$

Thus the prefix $a \in \{0, 1\}^m$ encodes a truth assignment. The remaining $r + 1$ positions are then determined by whether a satisfies φ .

If $a \models \varphi$, then for positions $m + 1, \dots, m + r$ the model emits the padding token p with probability 1, and at position L it emits eos with probability 1. Hence each satisfying assignment produces the unique complete sequence

$$a p^r \text{eos}$$

with probability 2^{-m} .

If $a \not\models \varphi$, then for positions $m+1, \dots, m+r$ the model emits b_0 and b_1 with probabilities $\frac{1}{2}$ each, and at position L it emits **eos** with probability 1. Hence each non-satisfying assignment a and each $u \in \{b_0, b_1\}^r$ produce a complete sequence

$$a u \text{eos}$$

with probability $2^{-(m+r)}$.

All complete sequences therefore lie in R_L . If φ is satisfiable, then

$$\text{AS}_L(P_\varphi) = \frac{m}{L},$$

achieved exactly by the sequences $a p^r \text{eos}$ corresponding to satisfying assignments. Any non-satisfying completion has average surprisal

$$\frac{m+r}{L} = \frac{m}{L} + \frac{r}{L}.$$

If φ is unsatisfiable, every completion has average surprisal $(m+r)/L$.

Since $r = cm$ and $L = (c+1)m + 1$,

$$\frac{r}{L} = \frac{cm}{(c+1)m + 1} \geq \frac{c}{c+2} > \varepsilon.$$

Therefore, in the satisfiable case, any additive- ε approximation algorithm must output a sequence of the satisfying form $a p^r \text{eos}$, because every non-satisfying completion is more than ε bits per token worse than optimum. In the unsatisfiable case, no such output exists. Thus SAT is decided by running the approximation algorithm and checking whether the returned sequence contains the padding block p^r . Because c depends only on the fixed constant ε , the reduction is polynomial. Hence the approximation problem is NP-hard. $\square$

9 Support coverage and reachability

To speak precisely about “which part of the solution space is explored,” it is useful to distinguish the support of the exact conditioned model from the support induced by a search procedure.

Definition 3 (Conditioned support and reachable support). *Fix an event $E \subseteq \mathcal{C}$. The exact conditioned support is*

$$S_\theta(E) = \{x \in E : P_\theta(x) > 0\}.$$

Let A be a possibly randomized constrained-generation procedure that, given (θ, E) , outputs either a sequence in E or a failure symbol $\perp$. Its reachable support under E is

$$R_A(\theta, E) = \{x \in E : \Pr[A(\theta, E) = x] > 0\}.$$

We say that A has exact support coverage on E if $R_A(\theta, E) = S_\theta(E)$.

Bias relative to the exact conditional law can then arise in two logically distinct ways. One may have *weight distortion*, where $R_A(\theta, E) = S_\theta(E)$ but the output probabilities do not match $P_\theta(\cdot \mid E)$. Or one may have *support loss*, where $R_A(\theta, E) \subsetneq S_\theta(E)$, so some admissible completions are never produced at all.

We do not claim that support loss occurs for every pair (θ, E) . Some events, such as fixed-prefix constraints, are exactly tractable, and bounded-state models admit exact dynamic programs for regular constraints. The point is instead a worst-case impossibility statement: over the broad autoregressive model class considered here, even simple regular positional events already defeat any generic polynomial-time guarantee of exact support coverage.

Proposition 1. *Let*

$$R_L = \{x \in \mathcal{C} : |x| = L\}.$$

Unless $P = NP$, there is no deterministic polynomial-time procedure A such that, for every succinct autoregressive model P_θ and every integer L , whenever $S_\theta(R_L) \neq \emptyset$ the procedure returns some sequence in $S_\theta(R_L)$.

Proof. Suppose such a procedure existed. Given a CNF formula φ over m variables, construct the model P_φ from the proof of Theorem 2 and set $L = m + 1$. In that construction,

$$S_{P_\varphi}(R_L) = \{a \text{ eos} : a \models \varphi\}.$$

Hence $S_{P_\varphi}(R_L) \neq \emptyset$ if and only if φ is satisfiable. Running A on (P_φ, R_L) would therefore decide SAT in polynomial time, a contradiction unless $P = NP$. $\square$

Remark 3. *Many practical heuristics are prefix-pruning searches: once a prefix h is discarded, no completion extending h can ever be returned. For an event E , if*

$$\sum_{y: hy \in E} P_\theta(hy) > 0,$$

then h has a feasible continuation under E . Pruning such a prefix makes every admissible completion extending h unreachable, so that part of $S_\theta(E)$ lies outside $R_A(\theta, E)$. Avoiding this kind of support loss generically would require certified continuation information, precisely the type of global quantity shown above to be hard to compute exactly.

Remark 4 (Immediate support loss for truncating decoders). *The existence of unreachable valid sequences is especially transparent for decoders that assign zero probability to some locally possible tokens, such as greedy decoding, top- k , top- p , or beam pruning. For any fixed k , one can construct a succinct autoregressive model and use the regular event*

$$R_2 = \{x \in \mathcal{C} : |x| = 2\}$$

*such that $S_\theta(R_2) \neq \emptyset$ but top- k reaches none of it. Indeed, let the first-step distribution assign positive probability to $k + 1$ tokens $a_1, \dots, a_k, a_{k+1}$, with a_{k+1} ranked $(k + 1)$ st. If the first token is a_{k+1} , the model emits **eos** next, producing a valid completion in R_2 . If the first token is any of $a_1, \dots, a_k$, the model forces a continuation of length different from 2. Then the exact conditioned support under R_2 is nonempty, but top- k deletes the unique feasible first move and therefore has empty reachable support on R_2 . The same logic applies to any truncating decoder: once a necessary token is assigned probability 0 at some prefix, every admissible completion through that token is unreachable.*

10 Tractable and intractable constraints

The support distinction above does not mean that every constraint makes sampling or decoding hard. The correct distinction is between constraints that can be enforced from a bounded state and constraints whose exact enforcement requires nontrivial global continuation masses.

A first easy case is prefix conditioning. If one conditions on a fixed prefix,

$$x_{1:k} = u,$$

then exact sampling is immediate: one starts generation from the prefix u and continues autoregressively. More generally, any constraint determined entirely by a bounded prefix can be enforced online without global renormalization.

A second positive case is when the model itself admits a bounded sufficient state.

Proposition 2. *Suppose the autoregressive model admits a finite state set S such that, at each position, the next-token distribution depends on the past only through the current state in S . This includes first-order Markov models as a special case. Let $R_L \subseteq \mathcal{C}$ be a regular set of complete sequences of length L , given by a deterministic finite automaton. Then the constrained normalization constant*

$$Z_{R_L}(\theta) = \sum_{x \in R_L} P_\theta(x),$$

exact sampling from $P_\theta(\cdot \mid x \in R_L)$, and exact MAP decoding over R_L are all computable in time polynomial in L , $|S|$, and the size of the automaton.

Proof. Let Q be the automaton state space. At position i , the pair $(s_i, q_i) \in S \times Q$ is a sufficient statistic for the constrained continuation problem: s_i summarizes the model’s dependence on the prefix, and q_i summarizes progress with respect to the regular constraint. A forward recursion on the product state space computes the total mass of all prefixes reaching each product state; backward values give exact continuation masses and hence exact conditional sampling; replacing sums by maxima yields Viterbi decoding on the same product graph. This is the standard bounded-state dynamic programming principle underlying exact sampling for regular and Markov constraints in finite-state models [11, 23]. $\square$

At the opposite extreme are global constraints such as exact terminal position, exact syllable count, or general sentence-level optimization. For such events, exact conditional sampling at time i requires probabilities of the form

$$\Pr_\theta(E \mid x_{1:i-1}a),$$

namely the probability that a partial continuation can still be completed into a full sequence satisfying the global event E . These are continuation masses over exponentially many suffixes. In the general autoregressive setting, the model class supplies no generic bounded sufficient statistic for these quantities, which is why exact conditioning becomes hard.

Proposition 2 shows that regularity of the constraint language is sufficient once the model also has a bounded state. For a general autoregressive model, however, the automaton state of the regular language is not enough: one would also need a bounded summary of the model’s dependence on the entire prefix. Our hardness results show that no such general reduction exists in the broad succinct autoregressive setting.

Thus the right boundary is not simply between constrained and unconstrained generation. It is between constraints that admit an exact bounded-state recursion together with the model, and constraints that require globally summing or optimizing over future continuations.

11 Inpainting as mixed prefix-suffix conditioning

Inpainting provides another natural illustration of the same phenomenon. Filling a blank in a partially specified sequence amounts to sampling under both a prefix constraint and a suffix constraint. The prefix part is compatible with autoregressive generation: conditioning on a fixed prefix simply means starting generation from that prefix. The suffix part is different. It amounts to conditioning on a future event, and therefore requires exact continuation masses over all ways of completing the missing segment so as to match the prescribed suffix.

This perspective also helps interpret existing infilling and inpainting systems. In symbolic music, closely related tasks include future-aware constrained generation and musical inpainting systems such as Anticipation-RNN, score inpainting models, the Piano Inpainting Application, and DeepBach [13, 14, 15, 17]. On the text side, fill-in-the-blank and text-infilling formulations likewise condition on both left and right context rather than only on a causal prefix [16].

Formally, let a partially specified sequence be of the form

$$u y v,$$

where u is a fixed prefix, v a fixed suffix, and y the missing segment to be generated. Exact inpainting requires sampling from the conditional distribution

$$P_{\theta}(y \mid u y v \in \mathcal{C}).$$

The fixed-length terminal problem studied above is an immediate special case: take $u = \epsilon$ empty, let $v = \text{eos}$, and require the total length to be L . Then inpainting reduces exactly to sampling from

$$P_{\theta}(x \mid x \in \mathcal{C}, |x| = L),$$

whose normalization constant was shown above to be $\#P$ -hard to compute.

Corollary 6. *Any generic exact inpainting procedure for succinct autoregressive models that samples from the true conditional distribution*

$$P_{\theta}(y \mid u y v \in \mathcal{C})$$

by causal left-to-right renormalization of a fixed pretrained autoregressive model must compute $\#P$ -hard conditioned normalization terms. In particular, exact inpainting does not reduce to ordinary left-to-right sampling from a fixed pretrained causal model by polynomial-time local reweighting alone.

Proof. Take the special case $u = \epsilon$, $v = \text{eos}$, and fixed total length L . Then exact inpainting contains, as a special case, conditioning on the regular language of complete sequences of length exactly L discussed in Section 5. The associated normalization constant is $Z_L(\theta)$, whose exact computation is $\#P$ -hard by Theorem 2. Therefore any causal exact sampling scheme based on local renormalization of next-token probabilities evaluates $\#P$ -hard continuation masses in the general case. $\square$

This gives a clean interpretation of current inpainting proposals. Fast practical systems avoid the exact-conditioning problem rather than solve it head-on. Anticipation-RNN explicitly replaces exact Bayesian conditioning of a fixed unconstrained model by a separately trained constrained model; text-infilling systems likewise train or fine-tune models specifically for infilling; the Piano Inpainting Application uses a dedicated encoder-decoder architecture; and DeepBach uses pseudo-Gibbs sampling rather than exact causal conditioning [13, 16, 15, 17]. These methods are approximate inference procedures or changes of model class, not generic exact samplers for the conditional distribution induced by an arbitrary pretrained autoregressive model. Equivalently, with the original causal model held fixed, they produce biased samples relative to the exact conditioned target law.

12 Discussion

The complexity statements above have a natural interpretation for contemporary language models. Exact rhyme, exact syllable count, and positional letter or suffix constraints are all examples of global form constraints on complete sequences. A request such as “produce a line of exactly twelve syllables,” “make the final word rhyme with *time*,” “ensure that the last word ends in *-ing*,” or “make the third word end with the letter *e*” is not hard to state, but exact satisfaction requires reasoning about future completions rather than only about local next-token likelihood.

This explains a familiar empirical phenomenon: autoregressive language models often come close to satisfying such constraints but miss by one syllable, produce an approximate rather

than exact rhyme, or satisfy one positional requirement only by violating another. Such failures are consistent with the complexity results proved here. In the general autoregressive setting, exact enforcement of global formal constraints requires access to continuation masses or global optimization procedures that are computationally intractable in the worst case.

The approximation result sharpens the same message. Even if one weakens the objective from exact optimum to *find a good constrained line*, the computational obstacle remains. Informally: among all Genesis-like continuations of exactly L tokens, even guaranteeing one whose average surprisal is within half a bit per token of the best valid continuation is already NP-hard in the general autoregressive setting.

The results do not imply that language models never satisfy rhyme, meter, or positional constraints. They do so approximately, and specialized external machinery helps substantially: constrained decoding [7, 8, 9], finite-state controllers, reranking, symbolic metrical modules, backtracking search, or dedicated infilling architectures in text and music [13, 14, 15, 16, 17] all impose structure that plain left-to-right sampling does not provide on its own. This paper stresses that exact satisfaction of such constraints does not arise generically from unconstrained local autoregressive sampling alone. When rhyme, meter, terminal patterns, or inpainting constraints are enforced without exact continuation masses, the resulting procedure is biased relative to the exact conditional distribution of the fixed underlying autoregressive model. These hidden inferential biases can appear either as wrong conditional weights on reachable completions or as support loss, where some admissible completions are never reached at all.

The strongest claims should be read in this scope. The paper proves worst-case intractability of exact conditioning for fixed autoregressive models and thereby rules out a generic exact polynomial-time constrained-sampling procedure across this model class. It does not provide a quantitative characterization of the magnitude of the induced bias for any specific practical system, nor does it imply that every constrained prompt exhibits a large empirical distortion. What it shows is that exact support coverage and exact conditional probabilities cannot be generically guaranteed without solving computationally hard continuation-mass problems. This is also the practically relevant regime for current transformer-based LLMs [6]: although deployed systems have finite precision and finite context windows, they do not come equipped with a tractable bounded sufficient state that would support the exact dynamic programs available in the first-order Markov case.

A similar caveat likely applies to diffusion models [18]. If

$$p_{\theta}(x_{0:T}) = p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1} | x_t)$$

is the reverse chain and E is a constraint on the terminal sample x_0 , then exact conditioning requires the guided reverse kernels

$$p_{\theta}(x_{t-1} | x_t, E) = p_{\theta}(x_{t-1} | x_t) \frac{h_{t-1}(x_{t-1})}{h_t(x_t)}, \quad h_t(x_t) = P_{\theta}(E | x_t),$$

which is the diffusion analogue of the continuation-mass correction in the autoregressive case. Practical guided samplers instead use classifier guidance or classifier-free guidance [19, 20], and posterior inference under diffusion priors has recently been explicitly framed as an intractable inference problem requiring approximate or amortized solutions [21]. Thus the proofs in this paper do not directly transfer to diffusion models, but the same inferential issue plausibly reappears there as well.

13 Conclusion

Autoregressive factorization provides local access to next-token probabilities, not a tractable general procedure for global mode-finding or exact conditioning under global formal constraints.

For succinctly represented autoregressive models with polynomial-time next-token evaluation, exact sentence-level MAP decoding is NP-hard, exact constrained MAP remains NP-hard under unary constraints and under a fixed metrical-count special case, and exact conditioned normalization is already #P-hard even for regular constraints such as fixed-length terminal events. Even approximate optimization remains hard in a strikingly strong form: for every fixed $\varepsilon < 1$, obtaining a feasible fixed-length completion within ε bits per token of optimum average surprisal is NP-hard. The positive side of the picture is the first-order Markov case: in such a case, constrained generation can be formulated exactly, and exact sampling with the correct conditional probabilities is available for regular, unary, and Markov constraints [22, 23]. Outside that bounded-state setting, the situation changes qualitatively. Whenever generation must satisfy exact global form constraints (rhyme, meter, terminal patterns, positional endings, or related formal devices) plain autoregressive generation faces a structural computational obstacle rather than a mere implementation inconvenience. Consequently, for the practical decoding procedures used with current transformer LLMs, constrained generation should be understood as approximate inference rather than exact conditioning. In practice these systems rely on heuristic local decoding and truncation [5], so there is no generic reason to expect recovery of the exact conditioned law or even consistently near-optimal high-probability constrained continuations, and admissible completions can be excluded from the search altogether.

References

- [1] Bryan Eikema and Wilker Aziz. Is MAP Decoding All You Need? The Inadequacy of the Mode in Neural Machine Translation. In *Proceedings of COLING 2020*, pages 3080–3090, 2020.
- [2] Su Lin Blodgett, Solon Barocas, Hal Daumé III, and Hanna Wallach. Language (Technology) is Power: A Critical Survey of “Bias” in NLP. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5454–5476, 2020.
- [3] Clara Meister, Tim Vieira, and Ryan Cotterell. If Beam Search is the Answer, What was the Question? In *Proceedings of EMNLP 2020*, pages 2173–2185, 2020.
- [4] Felix Stahlberg and Bill Byrne. On NMT Search Errors and Model Errors: Cat Got Your Tongue? In *Proceedings of EMNLP-IJCNLP 2019*, pages 3356–3362, 2019.
- [5] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The Curious Case of Neural Text Degeneration. In *International Conference on Learning Representations*, 2020.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems 30*, pages 5998–6008, 2017.
- [7] Chris Hokamp and Qun Liu. Lexically Constrained Decoding for Sequence Generation Using Grid Beam Search. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1535–1546, 2017.
- [8] Matt Post and David Vilar. Fast Lexically Constrained Decoding with Dynamic Beam Allocation for Neural Machine Translation. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1314–1324, 2018.
- [9] Ximing Lu, Peter West, Rowan Zellers, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. NeuroLogic Decoding: (Un)supervised Neural Text Generation with Predicate Logic Constraints. In *Proceedings of the 2021 Conference of the North American Chapter of the*

- Association for Computational Linguistics: Human Language Technologies*, pages 4288–4299, 2021.
- [10] Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D’Antoni. Grammar-Aligned Decoding. In *Advances in Neural Information Processing Systems 37*, 2024.
 - [11] Mehryar Mohri. Weighted Finite-State Transducer Algorithms: An Overview. In *Formal Languages and Applications*, pages 551–563. Springer, 2004.
 - [12] Pierre Roy and François Pachet. Enforcing Meter in Finite-Length Markov Sequences. In *Proceedings of the Twenty-Seventh AAAI Conference on Artificial Intelligence*, pages 854–861, 2013.
 - [13] Gaëtan Hadjeres and Frank Nielsen. Anticipation-RNN: enforcing unary constraints in sequence generation, with application to interactive music generation. *Neural Computing and Applications*, 32:995–1005, 2020.
 - [14] Ashis Pati, Alexander Lerch, and Gaëtan Hadjeres. Learning to Traverse Latent Spaces for Musical Score Inpainting. In *Proceedings of the 20th International Society for Music Information Retrieval Conference*, pages 343–351, 2019.
 - [15] Gaëtan Hadjeres and Léopold Crestel. The Piano Inpainting Application. arXiv:2107.05944, 2021.
 - [16] Chris Donahue, Mina Lee, and Percy Liang. Enabling Language Models to Fill in the Blanks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2492–2501, 2020.
 - [17] Gaëtan Hadjeres, François Pachet, and Frank Nielsen. DeepBach: a Steerable Model for Bach Chorales Generation. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1362–1371, 2017.
 - [18] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep Unsupervised Learning using Nonequilibrium Thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 2256–2265, 2015.
 - [19] Prafulla Dhariwal and Alexander Nichol. Diffusion Models Beat GANs on Image Synthesis. In *Advances in Neural Information Processing Systems 34*, pages 8780–8794, 2021.
 - [20] Jonathan Ho and Tim Salimans. Classifier-Free Diffusion Guidance. arXiv:2207.12598, 2022.
 - [21] Siddarth Venkatraman, Moksh Jain, Luca Scimeca, Minsu Kim, Marcin Sendera, Mohsin Hasan, Luke Rowe, Sarthak Mittal, Pablo Lemos, Emmanuel Bengio, Alexandre Adam, Jarrod Rector-Brooks, Yoshua Bengio, Glen Berseth, and Nikolay Malkin. Amortizing Intractable Inference in Diffusion Models for Vision, Language, and Control. In *Advances in Neural Information Processing Systems 37*, 2024.
 - [22] François Pachet and Pierre Roy. Markov Constraints for Controlled Sequence Generation. To appear in *Festum-PI 2025 Proceedings, Lecture Notes in Mathematics*, Springer, N. Alikakos and Cédric Villani, editors, 2026.
 - [23] Alexandre Papadopoulos, François Pachet, Pierre Roy, and Jason Sakellariou. Exact Sampling for Regular and Markov Constraints with Belief Propagation. In *Principles and Practice of Constraint Programming*, pages 341–350, 2015.

- [24] Festum-Pi. Konstantinos Daskalakis (MIT): AI, Game Theory, and Shakespeare. Video page for the Festum-Pi 2025 lecture by Constantinos Daskalakis, Chania, Crete, 15 July 2025. Posted 4 November 2025. <https://www.festum-pi.eu/post/kostis-daskalakis-mit-ai-game-theory-and-shakespeare>.